Protecting SaaS against REA assisted inspection

Measured controls and reproducible findings from an owned local war game

18 October 2026 | Updated research white paper | Baseline and session boundary studies

The strongest demonstrated protections were server-authoritative decisions, tenant authorisation, and removing unnecessary implementation details from client code and responses. REA continued to inspect the authorised user interface and API. These controls prevented specific unauthorised outcomes and reduced direct disclosure; this experiment did not establish that a SaaS product can be made impossible to analyse or imitate.

We executed REA 6.7.0 on a Windows laptop. The baseline compared unsafe and hardened versions of a synthetic pricing feature. Follow-up work added controlled variants, repeated probes, an end-to-end reproduction run and a new session-protected fixture. Across those stages, 56 measured REA commands completed and 23 application tests passed. Pages 11-14 report the added hard-block evidence and practical guidance.

Measured outcome	Unsafe	Hardened
Pricing formula in captured frontend	Readable	Not observed
Quote response at quantity 50	9 fields; 370 bytes	3 fields; 80 bytes
Legitimate quote total	GBP 1104	GBP 1104
Forged client total of GBP 0.01	Accepted	Rejected with 422
Tenant B reads tenant A quote	Allowed	Denied with 404
REA can inspect UI and API call	Yes	Yes

Read this table as the original baseline, not a production safety claim. The original hardened fixture still published a demo token; a later negative control confirmed that replaying it grants tenant-A access. The new session fixture removes that defect and demonstrates specific denials through REA. All browser scenarios retain incomplete-coverage flags. No independent blind prediction score or production certification is claimed. [E1-E8; follow-up evidence on pages 11-12]

What was tested and how evidence was handled

The target was an owned educational SaaS fixture bound to loopback. The operator used only synthetic accounts demo-A and demo-B. Ordinary customer observations, bounded quote requests and non-destructive synthetic mutations were in scope. No commercial service or external tenant was tested.

Component	Recorded configuration
Laptop	Windows x64 10.0.26300; Intel i7-8750H; 12 logical processors; 32 GiB RAM
Toolchain	Node 24.19.0; REA 6.7.0; pnpm 11.25.0
Browser	Google Chrome 155.0.8059.82; isolated headless profiles
Baseline targets	127.0.0.1:3000 unsafe; 127.0.0.1:3001 hardened
Capture policy	Request and response bodies; DOM and accessibility text; header values disabled
V8 coordination	Separate 10-second windows; click after REA armed notification

Three evidence classes

Direct REA evidence: browser scenarios, passive script inspection, bundle analysis with maps disabled and enabled, direct listener inspection, script export, static JavaScript analysis and V8 observations. Every measured command has a timestamped argument record and retained JSON output. All 18 exited zero. [E1-E6]

Supplemental application evidence: nine regression checks and direct HTTP responses on fresh instances of the unchanged lab. Four ordinary sample quantities were identical in both modes. A separate hardened fifth request tested quota enforcement. Authority and stream checks used separate fixture instances to avoid quota interference. These are not REA captures. [E7-E8]

Unmeasured claims: independent agent reconstruction, production deployment, decoy efficacy, distributed quotas, long-lived streams, user friction and commercial cloning cost. The same operator saw both conditions and white-team source. That precludes a valid blind inference comparison.

Coverage matters more than the exit code

Both scenarios retained 15 events and dropped zero, yet reported overall completeness as incomplete with events missing. The specific script and quote bodies used here were captured; GET request-body states were not_exposed. We retain the original flags. Playwright response-body bytes are API results and are not guaranteed to be bytes from the original wire transaction. Passive inspection also cannot recover network activity from before attachment. [E1]

Priority one enforce decisions on the server

Protect transaction integrity even when the customer understands the algorithm. The server must calculate or validate consequential values using trusted state. A browser may request a quote but must not choose its final price, tenant, permission set or approval status.

Measured result

For quantity 50, the unsafe endpoint accepted clientTotal 0.01 and returned HTTP 201 with a GBP 0.01 quote. The hardened endpoint rejected the same extra field with HTTP 422. A normal request returned GBP 1104 in both variants. A client-supplied tenantId also produced a tenant-B quote in unsafe mode and was rejected with 422 in hardened mode. [E7, controls 0, 7 and 8; E8, T02 and T09]

Implementation pattern

Authenticate the caller on the server. Parse a strict request schema allowing only customer-selected inputs such as productId and quantity. Reject unexpected authority-bearing fields. Validate the product, range and entitlement. Calculate the price from server-held rules and attach the tenant from trusted identity. Save the authoritative quote before returning a minimal response. Apply the same authority at later checkout, export and fulfilment stages.

For production, define the price lifetime and rule version, use decimal or integer minor-unit arithmetic appropriate to the currency, and use idempotency for retryable operations. Where the browser presents a quote identifier later, resolve it to server state and recheck ownership and validity. These are implementation recommendations beyond the toy fixture, not measured production features.

Acceptance checks

Submit ordinary input and verify the expected outcome. Add price, discount, role and tenant fields individually; each must either be rejected by the contract or be demonstrably ignored while the server computes the same authorised result. Reuse, expire and replay identifiers under explicit business rules. Verify that all equivalent entry points enforce the same checks. Record expected and actual status, persisted state, and a legitimate user control case.

Residual exposure and cost

Server authority stops a tested manipulation; it does not conceal the total returned to the customer. An analyst may reconstruct some pricing behaviour and still be unable to alter your records. Moving calculation server-side adds availability and latency dependencies, rule-version management and retry handling. Measure those costs before broad rollout.

Evidence strength: directly observed bounded HTTP outcomes plus passing regression checks. No payment processor, real billing lifecycle or production authentication was tested. The fixture uses publicly embedded synthetic bearer strings and is unsuitable as an authentication reference. [E7-E8]

Priority two enforce tenant boundaries on every path

Make authorisation independent of what REA can discover. Knowing a route, identifier or source function must not grant permission. Enforce the caller, object, requested action and tenant relationship at the point of access, including streams and replay.

Measured result

Tenant B retrieved tenant A's quote and its event with HTTP 200 in unsafe mode. Hardened mode returned HTTP 404 for both. The ordinary tenant-A status request still returned HTTP 200. These outcomes were recorded against the exact newly created synthetic quote identifiers. [E7, controls 1-3]

The hardened fixture derived tenant identity on the server and checked the quote's tenant before serialisation. It also denied any Last-Event-ID replay request with HTTP 409. That is a measured refusal of unsupported replay, not proof of secure retained-history replay. [E7, control 4]

Implementation pattern

Scope data access by the authenticated principal and tenant in one authoritative layer. Apply explicit permissions to reads and writes. Test detail routes, search, bulk export, background jobs, caches, file links and event delivery. An unpredictable object identifier is useful hygiene but cannot replace the permission check. OWASP API1 recommends object-level checks wherever client input selects a record. [S2]

For real streaming systems, authorise subscriptions before attaching them, scope every emitted payload, and re-evaluate permission changes. Bind replay cursors to the tenant and subscription. Reconnects, buffered messages, cached events and permission revocation require their own tests. These paths were not modelled by the finite SSE fixture.

Acceptance checks

Create an object as A. Access it as A, B and an unauthenticated caller through each supported path. Confirm the expected minimal A response and absence of object content for B. Repeat after ownership or entitlement changes. Verify that denials do not create or mutate records. Confirm that logs show the denial without storing credentials or private response bodies.

Trade-offs and limits

A 404 response can avoid revealing whether an inaccessible object exists, but status choice alone provides no authorisation. Consistent tenant filtering requires careful cache keys and background-worker context. Logging and incident review add operational work. This experiment used one process and two fixed synthetic identities; it did not establish policy completeness or cross-service isolation.

The hardened measurement recorded tenant_boundary_denied and event_boundary_denied events with actor and route metadata. This demonstrates the existence of useful signals in the fixture, not the effectiveness of a production alerting programme. [E7, monitoring]

Priority three remove direct implementation disclosures

Keep protected decisions out of delivered code and explanatory payloads. Server-only logic, minimal response schemas, private source maps and removal of debug routes work together. Moving code alone is insufficient if responses still publish the rules.

Measured code and response differences

REA captured the entire unsafe 1147-byte application script, including baseUnitPrice 24, three volume bands and calculateInBrowser. The hardened 537-byte script contained the input, request and rendering code without that pricing formula. REA still found its /api/quotes endpoint and click handler. [E2-E3]

The quote body at quantity 50 changed from nine fields to three. Hardened returned quoteId, total and currency. Unsafe additionally returned tenant, productId, quantity, ruleId, threshold and discountBands. Captured body sizes were 370 and 80 bytes respectively. These are measurements of this JSON instance, not a security percentage or a throughput result. [E1]

The finite unsafe event exposed tenantId, ruleId and threshold. Hardened returned quote.ready with quoteId and status. Its debug rule route and source-map route returned 404, while unsafe returned both. Event and route outcomes came from supplemental HTTP measurements. [E7, controls 2, 5 and 6]

Source maps are a secondary control

With map fetching enabled, REA obtained a 368-byte unsafe map and 257 bytes of embedded source under src/quote-engine.ts. The map had empty mappings. This demonstrates original-name and source-content disclosure, not successful coordinate mapping or full original TypeScript recovery. The formula was already readable without fetching the map; removing maps alone would therefore leave the primary leak. [E2, E4]

Implementation and verification

Separate private calculation modules from frontend dependency graphs. Use explicit allowlists for response fields rather than serialising internal models. Review nested DTOs, errors, diagnostic headers and events. Keep debug routes unavailable to customer roles. Store production maps in a private diagnostic system if needed, and review old CDN artifacts and alternate asset paths. OWASP API3 supports property-level control and restrained response schemas. [S3]

Scan release artifacts and inspect the actual production delivery surface. The included regex scan passed hardened and intentionally failed unsafe; it recognises fixture-specific strings and is not a comprehensive secret scanner. Expand coverage to lazy chunks, workers, WASM, embedded configuration and caches. Removing customer-needed detail can harm support and accessibility; validate product requirements separately. [E8]

Priority four bound the oracle and measure what remains visible

Rate limits constrain query volume but cannot erase information in legitimate answers. The hardened fixture permitted four consecutive ordinary quote requests and rejected the fifth with HTTP 429. Each successful response still supplied an exact total. [E7]

Quantity	Unsafe total GBP	Hardened total GBP	Effective unit price
1	23.52	23.52	23.52
25	588.00	588.00	23.52
75	1656.00	1656.00	22.08
150	3168.00	3168.00	21.12

These samples reveal three effective unit prices. They do not determine the exact boundaries by themselves. Under a piecewise-constant unit-price assumption, transitions at 26 and 76 can fit the four pairs, as can transitions at 50 and 100. Other functional forms can fit too. This is an illustration of ambiguity, not an independent red-agent result or a lower bound on reconstruction difficulty.

Implementation pattern

Define a budget for the business action and account for authenticated identity, tenant and resource cost. Enforce it atomically across replicas and equivalent endpoints. Distinguish quota exhaustion from malformed input and transient failure. Supply useful retry behaviour, record the reason, and calibrate to ordinary customer tasks. OWASP API6 treats unrestricted automation of sensitive flows as a business risk requiring context-specific controls. [S4]

The current limiter is in memory, allows four attempts per user per 60 seconds, charges rejected submissions against that budget and does not return Retry-After. It is a synthetic demonstration. Persistence across restarts, replica consistency, concurrency and user-impact behaviour remain untested.

Detection and operational acceptance

Record quota exhaustion, rejected authority fields, tenant-boundary denials and debug-route attempts as structured metadata. Test whether the events reach an alerting or review workflow and how many legitimate sessions trigger it. Repeated boundary violations are different evidence from ordinary product inspection. Source downloads and low-volume quote requests alone do not identify intent.

Evaluate legitimate completion rate, retry delays, support tickets and false positives before tightening budgets. More restrictive access to exact outputs may reduce inference opportunities but can also change the product contract. No such commercial trade-off was measured here. The study does not show that four, ten or twenty well-chosen queries cannot reconstruct the hidden rule.

What the run does not support as a reliable blocker

The hardened product remained observable. REA associated a direct click listener with its source, exported and parsed that script, recorded the callback running, and retained the /api/quotes request initiator. The browser response also named CSP and related security headers. Their presence did not prevent these observations; the paired design does not isolate each header's individual effect. [E1-E6]

Technique	What this study establishes	Decision
Server authority and tenant checks	Tested forged values and cross-tenant reads were denied	Use as primary integrity controls
Server-only rules and minimal DTOs	Direct rule content absent from selected hardened captures	Use to reduce unnecessary disclosure
Private source maps	Original-source channel removed; source alone still leaked in unsafe	Supporting control
Rate limiting	Fifth request denied; four useful totals still visible	Sampling-cost control
CSP and browser headers	Hardened UI inspection still worked	Use for intended browser protections; no REA-blocking claim
Obfuscation, anti-debugging, fingerprint blocks, WAF rules	Not evaluated	No efficacy ranking from this experiment
Decoys and canaries	Not evaluated	Research hypothesis only

Why there is no universal winner

The word stop covers different goals. Preventing an unauthorised quote mutation is a binary outcome in this test. Reducing readable pricing code is a disclosure result. Making imitation more expensive requires an inference and economics study. These cannot be collapsed into one REA blocking percentage.

An authorised browser receives code, UI state and permitted outputs so it can function. Preventing inspection of that material was not demonstrated. Browser-side obfuscation or detection would need separate, repeatable tests of analyst effort, legitimate breakage and false positives before it could be recommended as an effective additional control.

For optional decoy research, keep the module inert and separate from real prices, identity and permissions. Randomise independent agent sessions, freeze prompts and budgets, and measure false inferences and customer impact. Do not silently return false customer prices or use prompt injection as a defence. No decoy result is claimed in this paper.

Adoption guide and repeatable acceptance criteria

Apply the controls to a concrete feature and its data flows. The ordering below prioritises unauthorised outcomes first, then disclosure reduction and sampling cost. It is an engineering sequence supported by a small fixture, not a measured ranking across all SaaS architectures.

No.	Engineering work	Evidence required before rollout
1	Map trust boundaries and enforce server authority	Forged price, tenant and role inputs rejected; normal action succeeds
2	Authorise objects, streams, jobs and caches	A/B/unauthenticated access matrix passes through every equivalent path
3	Remove private code and reduce DTOs	Delivered artifacts lack protected implementation; schema tests permit only required fields
4	Restrict maps, debug routes and stale artifacts	Customer requests cannot retrieve selected private artifacts; support workflow remains usable
5	Add atomic budgets and structured monitoring	Concurrent quota tests; useful retry behaviour; denial signals and legitimate-flow checks
6	Run independent reconstruction trials	Frozen red model, enforced isolation, unseen holdouts and explicit error metrics

Build an evidence record that others can challenge

Retain the fixture version, package integrity, browser version, complete commands, action timestamps, successful and failed requests, and raw coverage flags. Link every assertion to a file and a JSON path. Use OBSERVED, NOT_OBSERVED_IN_CAPTURE, UNKNOWN_INCOMPLETE, ERROR, CONTROL_PREVENTED or HYPOTHESIS_ONLY consistently. A missing capture or failed tool must never be scored as a defence.

Operational measures beyond this lab

Measure customer task success, p50/p95 latency, support impact, accessibility, availability, alert volume and false-positive rate. Test rule and permission changes, deploy rollbacks, retry behaviour and multi-region consistency. Exact performance numbers from a single headless toy flow would not answer those questions.

How to reproduce the capture stage

The sharing kit includes the unchanged lab, pinned package metadata, scenarios, command records, SHA-256 manifests and a portable reproduce.mjs. Install the exact REA version and supply an absolute Chrome-family executable path and a fresh run ID. The portable runner uses ephemeral loopback ports and a native CDP mouse click after the armed notification; the original laptop run used ports 3000/3001 and a Playwright action coordinator. The method difference is explicit. Follow the kit README for commands.

Evidence index and research limitations

Evidence files are relative to the run directory. Shared copies replace local account and workspace paths and are labelled derived; original files remain local. The hash mapping records original and shared digests. The 39-row observations.csv provides assertion-level status and paths; commands.jsonl records exact measured REA invocations.

ID	Evidence file and location	Supports
E1	rea-evidence/*-scenario.json normalized_result.events.items[14].body; steps[2].artifacts; completeness	Quote bodies, DOM outcomes and capture limits
E2	rea-evidence/*-inspect.json normalized_result.scripts.items[0].source.artifact	Retained client source
E3	rea-evidence/*-bundle.json and *-listeners.json observations.endpoints; listeners[0] within normalized_result	Endpoint and callback observations
E4	rea-evidence/*-bundle-maps.json normalized_result.observations.source_maps	Optional map fetch and embedded source
E5	rea-evidence/*-v8.json normalized_result.coverage.scripts and requests[0]	Application execution and initiators
E6	rea-evidence/*-export-manifest.json and *-js-analysis.json normalized_result.scripts; statistics	Exported script identity and inert static analysis
E7	white_only/api-controls.json modes.*.samples, controls, monitoring; hardened.fifth	Supplemental HTTP controls and sampling
E8	logs/node-test.txt and scan-*.txt	Nine passing regressions and artifact scans

Interpretation limits. This is a three-band toy function with static demo credentials, one server process and a finite SSE response. Browser scenarios are incomplete overall. V8 instrumentation changes timing and includes action-tool scripts; only the exact app.js URL is used for application counts. Initiator association has causal_attribution unknown. Worker, frame and WASM coverage is outside the demonstrated case. Static parsing is not execution and source-map sourcesContent is not full source recovery. [E1-E6]

Experimental limits. One laptop, browser and paired fixture cannot quantify broad efficacy. Controls changed together, without factorial ablation. No independent red session was isolated, no valid frozen-model holdout score exists, and no production-scale or user-impact study was run. The results support narrow architecture choices and a continuation foundation. They do not support claims that REA was universally stopped.

Independent continuation and sources

The next experiment

A separate white operator should create an OS-enforced red environment containing only one condition's approved observations. Do not give the hardened red agent this paper, the unsafe capture, backend, tests, scorer or earlier transcripts that reveal the rule. Restrict oracle access to a logged, equal budget and count scenario-created quotes as queries.

Freeze the red model, reasoning and hashes before revealing holdout quantities. Generate fresh white-only holdouts after the freeze; the supplied kit's fixed public list should not be repeatedly treated as unseen. Close oracle access before prediction. Score coverage, exact-match fraction, mean absolute error, maximum error and each pricing band. Blank predictions count as non-matches, not zero error. This is the missing stage needed to quantify independent replication.

Useful follow-on fixtures include valid minified source maps, cached old bundles, workers, WASM, error DTOs, permission changes during streaming and distributed quota enforcement. Change one factor at a time where possible. Publish failures, unchanged outcomes and cases where a proposed defence harms customers. Preserve legitimate inspection for security testing, accessibility and interoperability.

Sources and provenance

S1. REA official repository and installed rea-agents 6.7.0 package. CLI help and the installed distribution define the executed interfaces. The earlier protocol cited commit 32f0e5120966be59d17a1e092353816c76a9c5e2; no equivalence between that commit and npm 6.7.0 is asserted. Repository: github.com/morluto/rea. Package: rea-agents 6.7.0.

S2. OWASP API1 2023 Broken Object Level Authorization. Supports checking the authenticated caller's access to each selected object. Official OWASP API1 guidance.

S3. OWASP API3 2023 Broken Object Property Level Authorization. Supports controlled writable properties and intentional response schemas. Official OWASP API3 guidance.

S4. OWASP API6 2023 Unrestricted Access to Sensitive Business Flows. Supports context-specific treatment of excessive automated business actions. Official OWASP API6 guidance.

External sources were checked on 18 October 2026. They support architecture context, not the local experiment's numerical results. The local files E1-E8 are the primary evidence for those results. The supplied protocol version 1.1 and reproducibility archive define the starting fixture; their earlier results are not relabelled as this run.

The preceding pages describe the immutable baseline. The following pages add the later controlled experiments and session-protected fixture. Independent blind reconstruction and broader efficacy studies remain open.

Hard blocks demonstrated through REA

A hard block here means the server refused a defined request and returned no protected result. It does not mean the REA process stopped running. The new lab-v2 fixture makes the refusal depend on server-held identity, permission and state, rather than a tool name or browser fingerprint.

Attempt or control	REA observed result	Meaning
No session	401 on protected probes	Authentication required
Forged session cookie	401 on protected probes	Inventing a cookie does not grant access
Replay of a revoked session	401 on protected probes	Server revocation is enforced
Wrong password	Login 401; protected probes 401	Failed authentication grants no access
Tenant B reads A quote or event	404 for both	Tenant boundary enforced
Read-only user creates quote	403	Write permission enforced
Client price or tenant field	422 for both	Client cannot choose trusted values
Missing CSRF token in valid session	403	Request-integrity requirement enforced
Fifth valid quote in one-minute budget	429; Retry-After 60	Bounded action budget enforced
Authorised owner creates quote	201; GBP 1104	Legitimate function preserved

Nine REA browser scenarios exercised these conditions using fresh synthetic credentials, opaque sessions and an explicit research-probe UI. The fixture clears the password field after login. REA secret references redact the generated test passwords; cookie values are not captured as headers. Fourteen regression tests additionally check expiry, login rotation, foreign Origin, budget persistence across login, retry recovery and entitled status. [E10]

What remains accessible: authorised users can inspect the public UI script, learn the route shape, obtain their permitted totals and automate within the budget. A valid stolen session, compromised endpoint or server compromise is not defeated merely by these gates. The fixture demonstrates request-level denials, not exhaustive security.

Stronger evidence from isolated changes and repetition

Three controlled variants isolate common incomplete fixes. Each retains the unsafe backend except for the named change. REA captured a normal browser quote and the delivered JavaScript from each. No baseline files were edited. [E9]

Only change applied	What REA still obtained	Result
Remove source-map reference and map route	Client pricing formula and 9-field response with bands	Map removal alone fails to hide the rule
Replace pricing frontend with server-request UI; remove map	No client formula, but response still exposes all bands	Server-only UI alone fails if API discloses the rule
Reduce quote response to 3 fields	Full formula remains in delivered JavaScript	Response minimisation alone fails to hide the rule

Repeated blocking outcomes

Unsafe and hardened browser-driven control probes were repeated three times on fresh fixtures. In each hardened run, two cross-tenant reads returned 404 and two client-authority mutations returned 422; all corresponding unsafe requests succeeded. Across those repetitions, this is six denied cross-tenant reads and six rejected authority mutations in hardened mode, versus six successful reads and six accepted mutations in unsafe mode. Legitimate quote and status controls succeeded in all six runs. This is a small deterministic repetition count, not a population success rate. [E9]

Separate REA quota scenarios sent five requests for four distinct quantities. Unsafe accepted all five. Hardened accepted the first four and rejected the fifth. The original portable capture runner also completed an end-to-end repeat of all 18 baseline REA operations using fresh ports and profiles. Its V8 action uses native CDP mouse dispatch after the armed notification, avoiding the original action helper's injected-script noise. [E9, E11]

A negative control that changes the recommendation

In a direct HTTP test, tenant B's credential received 404 for A's quote. The demo-A token extracted from the public hardened-v1 script received 200 for that same quote. Therefore v1 is not safe as an identity design. Lab-v2 stores account verification and random session state on the server, publishes no demo bearer string, and rejects that legacy token when unauthenticated. [E9, baseline-public-token-replay.json; E10]

E9: runs/laptop-20261018-supplemental, complete REA JSON and summary.json. E10: runs/session-boundaries-20261018, nine REA JSON files and 14-test log. E11: runs/validation-20261018, 18 command records and captures. Scenario assertions can be checked in normalized_result.steps[-1].artifacts.dom.value.text and the linked network response bodies.

What each team should do

Developers own the enforcement and the tests

Implement strict input schemas, server-calculated decisions, explicit output schemas and object-level permission checks. Keep proprietary logic and credentials out of client assets. Use a maintained session and identity stack. Add negative tests for missing, forged, revoked and expired identity, another tenant's object, extra authority fields and invalid request-integrity tokens. Pair every denial test with an authorised success case.

Evidence to deliver: failed attack requests, unchanged authoritative state, absence of protected fields in the denial response, source and response artifact scans, and passing normal user flows. Rerun those checks on every change to identity, policy, pricing, exports or streaming.

Architects own complete and consistent boundaries

Identify which decisions and datasets must remain private, then place them behind server-enforced services. Inventory every route to the same capability, including jobs, caches, exports, streams and support tooling. Use tenant-scoped storage and cache keys, least-privilege service identities, central policy where appropriate, atomic distributed quotas, and revocation that propagates across replicas.

For a deployed service, add TLS, Secure cookies, robust key/secret management, account lifecycle controls and dependable audit pipelines. Test failure modes: stale permissions, cache confusion, replica races, unavailable identity dependencies and replay. Keep private source maps and diagnostics available to trusted operators without delivering them to customers.

Evidence to deliver: a trust-boundary diagram, route and data inventory, threat model, multi-tenant access matrix, revocation tests across replicas, operational failure tests and an independent review. None is replaced by blocking one client fingerprint.

Product managers own the information contract and acceptable friction

Decide which exact outputs customers genuinely need. Anything intentionally returned to an entitled user can inform that user about product behaviour. Reduce unnecessary explanatory fields, internal identifiers, detailed traces and bulk exports while retaining useful outcomes. Define who may run expensive or sensitive actions, at what volume, and how customers recover from legitimate quota exhaustion.

Evidence to deliver: explicit entitlement and output requirements, authorised task completion rate, latency and retry experience, support impact, false-positive rate, and a decision about residual inference risk. Do not turn the lab's small pass count into a sales claim that the product is REA-proof.

Security and operations should jointly test credential theft, abuse response, dependency patching and incident handling. A server-side deny rule protects only if it is consistently deployed, monitored and maintained. These production recommendations extend beyond the measured localhost fixture.

Protection claims that can be defended

Supported claim: under the stated synthetic identities and requests, server-enforced session, permission, tenant, schema and quota checks denied access or mutation, while an authorised quote still succeeded. Removing client rules and excess response fields measurably reduced direct disclosure. REA recorded those outcomes rather than bypassing the application's boundaries.

Unsupported claim: all applications using these patterns are safe against every REA-assisted attack, or authorised customers can no longer infer their permitted outputs. No finite run in this repository proves that. REA is an observation and analysis tool; the demonstrated boundaries apply to equivalent HTTP clients too.

How the success criteria are made reviewable

The repository includes a plain-English start page, a control guide, unmodified baseline lab, separate session lab, evidence index, acceptance checks, hashes, portable runners and this paper. The acceptance checker reads the retained captures and checks the claimed source, response and denial outcomes. It does not infer security from a successful CLI exit. Redacted sharing copies retain the original coverage limitations.

There are 56 measured REA commands: 18 baseline, 11 supplemental, 18 reproduction-validation and 9 session-boundary captures. All completed. The test logs contain nine original regression checks and fourteen session checks. These totals count operations and tests, not independent security discoveries or exhaustive attack coverage.

Deployment and research gaps remain explicit

Lab-v2 still uses localhost HTTP, synchronous password verification and in-memory sessions, quotes and quotas. It is not a hardened production identity service. A real deployment needs transport security, maintained authentication, durable and distributed enforcement, account recovery and lifecycle controls, secure operations, monitoring, load and usability tests, and independent adversarial assessment.

No independent blind reconstruction, adaptive autonomous attacker, commercial-scale cloning, long-lived stream revocation, distributed race test, WAF comparison, obfuscation study or decoy trial was completed. Capture completeness remains incomplete overall. Authentication failures are request-level proof in scope; they do not establish the absence of every bypass.

Practical decision

Use the session fixture and its acceptance criteria to understand hard request boundaries. Use the unsafe variants to demonstrate why partial secrecy measures fail. Carry the tests into your own feature, expand them to its real trust boundaries, and measure legitimate customer impact. Treat residual inference as a product and architecture question, not a promise that an inspection tool can be made blind.